

Architecture Design Document

Progetto: Music Player

Data: 28/05/2026

Candidati: Del Sorbo Palma, Falcone Michele, Avallone Emanuele, De Bonis Francesco

Link Repository Github: <https://github.com/EmanueleAvallone/SAD-Project->

1. Introduzione

Il presente documento descrive l'architettura software iniziale del progetto Music Playlist Manager, un'applicazione desktop Java per la gestione di tracce audio, playlist e playback simulato.

L'obiettivo della progettazione architeturale è trasformare i requisiti funzionali e non funzionali in una struttura software coerente, suddivisa in componenti con responsabilità chiare.

L'architettura rappresenta il disegno di alto livello del sistema: definisce i principali elementi software, le loro proprietà visibili dall'esterno e le relazioni tra tali elementi.

Il documento ha quindi lo scopo di chiarire:

- quali componenti principali compongono il sistema;
- come tali componenti interagiscono;
- quali pattern architeturali e design pattern sono stati scelti;
- quali principi di buon design guidano la progettazione;
- come l'architettura supporta testing, manutenibilità ed evoluzione futura.

2. Scelta architetturale generale

L'applicazione sarà organizzata secondo un'architettura a livelli ispirata al pattern Model-View-Controller (MVC). Questa scelta è coerente con la natura del sistema, che richiede un'interazione continua tra utente, interfaccia grafica, logica applicativa e stato interno del player.

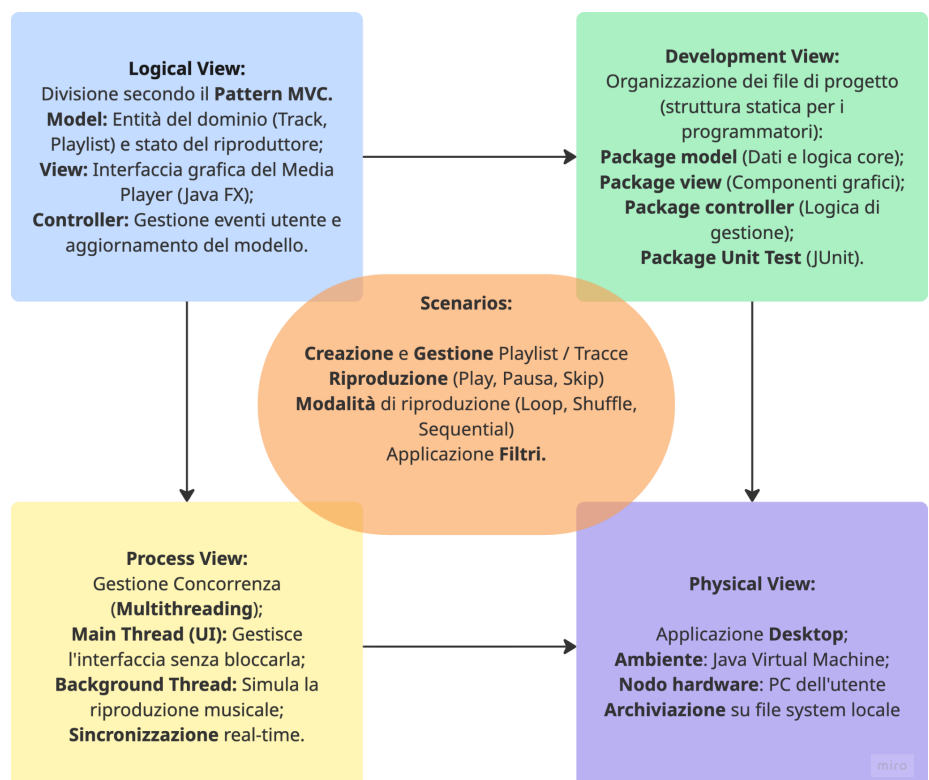
MVC è uno standard per gestire l'interazione tra utente e sistema, separando i dati dalla loro rappresentazione visiva. Il Model incapsula lo stato dell'applicazione e le funzionalità di business; la View si occupa della rappresentazione grafica; il Controller mappa le azioni dell'utente in aggiornamenti del modello.

La parte grafica sarà realizzata con JavaFX e file FXML generati tramite Scene Builder. I controller JavaFX non conterranno direttamente la logica di business, ma si limiteranno a ricevere gli input dell'utente e a delegare le operazioni ai servizi applicativi.

Questa scelta permette di mantenere separati i diversi interessi del sistema, riducendo l'accoppiamento tra interfaccia grafica e logica applicativa.

Questa soluzione supporta manutenibilità, testabilità, basso accoppiamento (cioè bassa interdipendenza tra moduli) e alta coesione (cioè moduli con responsabilità specifiche e unitarie), che sono i criteri fondamentali di un buon design software.

2.1 Modello 4+1 View



L'architettura è descritta attraverso il modello 4+1 View di Kruchten, che consente di rappresentare il sistema da più punti di vista, ognuna dedicata a uno specifico concern: Logical View, Process View, Development View e Physical View; la vista "+1" è costituita dagli Scenarios, cioè dai casi d'uso principali che guidano e validano l'architettura.

- La **Logical View** descrive la struttura funzionale del sistema e si concentra sulla decomposizione object-oriented e sulle astrazioni principali.

Nel nostro progetto, questa vista è organizzata secondo MVC:

- il Model contiene classi come Track, Playlist, Library, Player;
- la View contiene le schermate JavaFX/FXML;
- il Controller gestisce gli eventi utente e invoca i servizi;
- la Business Logic coordina le operazioni tra controller e modello.

Questa vista serve a chiarire le responsabilità logiche principali e a evitare che l'interfaccia grafica contenga direttamente la logica applicativa.

- La **Development View** mostra come il software è organizzato per lo sviluppo, quindi moduli, package e sottosistemi. È la vista più utile per sviluppatori e project manager. L'organizzazione progettata permette di dividere il lavoro tra i membri del team e facilita l'integrazione dei componenti durante gli sprint.
- La **Process View** riguarda gli aspetti di runtime, come concorrenza, sincronizzazione e distribuzione delle funzioni tra processi. È utile per ragionare su performance e disponibilità.

Nel nostro sistema l'applicazione è desktop e non distribuita, quindi la Process View si concentra soprattutto sulla separazione tra:

1. thread principale dell'interfaccia JavaFX;
2. logica di aggiornamento dello stato del player;

3. eventuale simulazione temporale della riproduzione.

La gestione del playback simulato non deve bloccare l'interfaccia. Per questo motivo, l'avanzamento della traccia e l'aggiornamento dello stato dovranno essere gestiti in modo tale da mantenere la GUI reattiva.

- La **Physical View** identifica i nodi hardware su cui il software viene eseguito e mappa gli elementi software su tali nodi.

Nel nostro progetto la Physical View è semplice: l'applicazione viene eseguita come programma desktop sulla macchina dell'utente. Non è prevista, nella prima versione, una distribuzione client-server né l'uso di un database remoto.

I dati potranno essere inizialmente gestiti in memoria e, se previsto nelle evoluzioni successive, salvati su file system locale.

- La vista degli **Scenari** rappresenta i principali casi d'uso che permettono di verificare se le altre viste architetturali lavorano correttamente insieme.

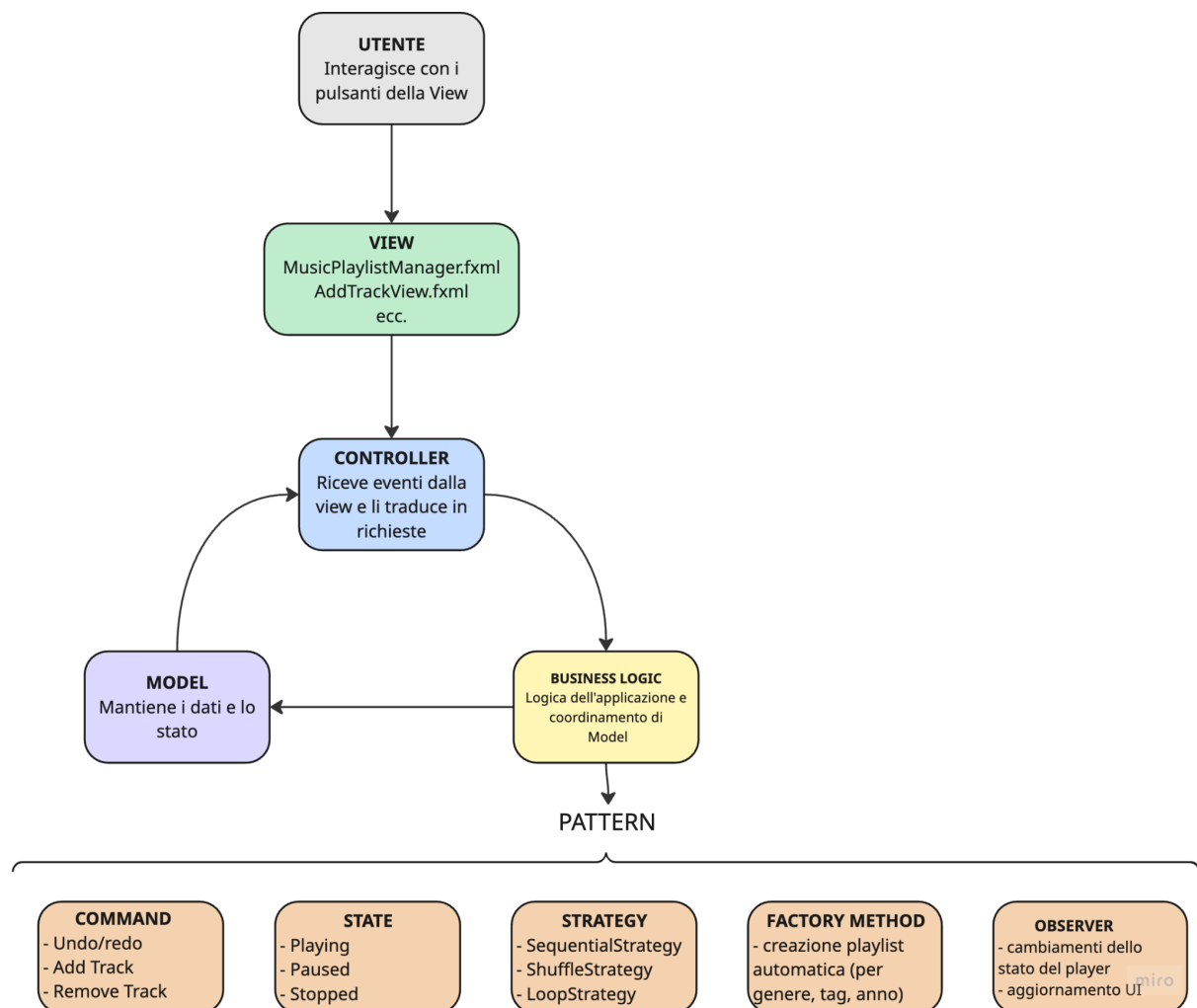
Gli scenari sono la vista basata sui casi d'uso e servono anche a scoprire nuovi elementi architetturali durante il design.

Gli scenari principali del nostro sistema sono:

- importazione di una traccia audio;
- modifica o eliminazione di una traccia;
- creazione di una playlist;
- aggiunta e rimozione di brani da una playlist;
- riproduzione, pausa, skip e previous;
- scelta della modalità di riproduzione;
- modifica della playlist durante la riproduzione;

- aggiornamento real-time dell'interfaccia;
- undo di operazioni di rimozione;
- generazione automatica di playlist per genere, anno o tag.

3. Struttura e pattern adottati



Lo schema rappresenta la struttura interna dell'applicazione secondo MVC e mostra come i principali design pattern vengono collocati all'interno dell'architettura.

3.1 Command Pattern

Il Command Pattern sarà utilizzato per rappresentare alcune azioni dell'utente come oggetti indipendenti.

Il Command è un pattern che trasforma una richiesta in un oggetto stand-alone contenente tutte le informazioni necessarie per eseguirla. È particolarmente utile per accodare, registrare richieste e supportare operazioni di undo.

Nel nostro progetto verrà usato per operazioni come:

- aggiunta di una traccia a una playlist;
- rimozione di una traccia;
- eventuale modifica di una playlist;
- undo di operazioni annullabili.

Il controller JavaFX sarà l'invocatore dell'azione, mentre il service sarà il receiver che conosce la logica applicativa. Ogni comando potrà essere salvato in una cronologia per permettere l'annullamento dell'ultima operazione supportata.

Questa scelta riduce l'accoppiamento tra interfaccia e logica applicativa, perché il controller non deve conoscere i dettagli interni dell'operazione.

3.2 State Pattern

Lo State Pattern sarà utilizzato per modellare lo stato del player simulato.

Lo State permette ad un oggetto di alterare il proprio comportamento quando cambia il suo stato interno, evitando lunghe catene di condizioni dipendenti dallo stato.

Nel nostro sistema il player potrà trovarsi, ad esempio, negli stati:

- Stopped;
- Playing;

- Paused.

Le operazioni play, pause, stop, next e previous avranno comportamento diverso a seconda dello stato corrente. Ad esempio, pause ha senso solo se il player è in riproduzione, mentre play può avviare una nuova traccia oppure riprendere una traccia sospesa.

L'uso dello State Pattern evita di concentrare tutta la logica del player in una sola classe piena di if o switch, rendendo il codice più leggibile e manutenibile.

3.3 Strategy Pattern

Lo Strategy Pattern sarà usato per gestire le diverse modalità di riproduzione.

Lo Strategy è un pattern che incapsula una famiglia di algoritmi in classi separate e li rende intercambiabili.

Nel nostro progetto le strategie principali saranno:

- SequentialStrategy;
- ShuffleStrategy;
- LoopStrategy.

Tutte avranno lo stesso obiettivo: determinare quale traccia deve essere riprodotta dopo quella corrente. Tuttavia, ciascuna applicherà una regola diversa.

Questa scelta consente di cambiare modalità di riproduzione a runtime senza modificare la classe principale del player. Inoltre evita la presenza di lunghi blocchi condizionali all'interno del player e favorisce l'estensione futura con nuove modalità.

3.4 Factory Method

Il Factory Method sarà utilizzato per la creazione automatica delle playlist.

Il Factory Method definisce un'interfaccia per creare un oggetto lasciando alle sottoclassi o a componenti dedicati la decisione su quale classe istanziare, riducendo la dipendenza da classi concrete.

Nel nostro sistema, factory potrà essere usato per creare playlist basate su:

- genere musicale;
- anno di pubblicazione;
- tag visivi;
- eventuali criteri futuri.

Il controller non dovrà conoscere la logica interna di filtraggio del catalogo: invierà alla factory il criterio scelto dall'utente, e la factory restituirà una playlist già costruita.

Questa scelta favorisce l'Open/Closed Principle, perché nuovi criteri di generazione potranno essere aggiunti senza modificare direttamente i controller.

3.5 Observer Pattern

L'Observer Pattern sarà utilizzato per mantenere aggiornati i componenti dell'interfaccia quando cambia lo stato del player.

L'Observer è un meccanismo di sottoscrizione che permette a un soggetto osservabile di notificare più oggetti osservatori quando il proprio stato interno cambia.

Nel nostro progetto il player svolge il ruolo di soggetto osservabile, mentre i controller dell'interfaccia possono agire come osservatori interessati ai cambiamenti. Quando il player cambia stato, aggiorna la playlist corrente, modifica la traccia

attuale o cambia modalità di riproduzione, gli osservatori registrati vengono notificati e possono aggiornare la parte grafica corrispondente.

I cambiamenti osservabili riguardano principalmente:

- cambio dello stato del player;
- passaggio tra Playing, Paused e Stopped;
- cambio della playlist corrente;
- cambio della traccia attualmente selezionata o riprodotta;
- cambio della modalità di riproduzione.

Questa scelta è coerente con MVC, perché il Model non deve conoscere direttamente la View. Il Model notifica un cambiamento, mentre il controller aggiorna l'interfaccia.

4. Pattern architetturali scartati

Sono stati valutati anche altri stili architetturali, ma non sono stati scelti come struttura principale.

- ❖ Lo stile **Pipe and Filters** è adatto a sistemi che elaborano flussi di dati attraverso trasformazioni sequenziali indipendenti, come compilatori o pipeline di analisi dati.

Non è stato scelto perché il nostro sistema non è principalmente una pipeline di trasformazione dati, ma un'applicazione interattiva desktop. L'utente deve poter creare playlist, modificare dati, controllare la riproduzione e ricevere aggiornamenti immediati dall'interfaccia.

- ❖ Lo stile **Shared Data** pone un deposito dati centrale, come un database, al centro del sistema ed è tipico delle applicazioni aziendali che gestiscono grandi quantità di dati persistenti.

Non è stato scelto come architettura principale perché nella prima versione del progetto il focus non è la gestione transazionale di un database, ma la gestione in memoria di tracce, playlist e stato del player.

- ❖ Lo stile **Client-Server** separa chi richiede un servizio da chi lo fornisce, mentre il modello Three-Tier divide Presentation, Application Layer e Database Server.

Questi stili non sono necessari nella prima versione perché l'applicazione è desktop e locale. Potrebbero diventare utili solo in un'evoluzione futura, ad esempio se si volesse sincronizzare la libreria musicale su cloud o condividere playlist tra utenti diversi.

5. Principi di design adottati

L'architettura proposta è guidata dai principi di buon design software studiati nel corso.

Il primo principio adottato è la **Separazione delle Responsabilità**. Ogni area funzionale viene gestita da un modulo distinto: la View gestisce la rappresentazione, il Controller gli eventi utente, il Model lo stato, i Service la logica applicativa. La teoria collega questo principio alla coesione, perché ogni modulo deve occuparsi di una sola cosa.

Il secondo principio è il **Single Responsibility Principle**: ogni componente deve svolgere un singolo compito ben definito. Per esempio, Playlist rappresenta una raccolta di tracce, mentre la logica per creare playlist automatiche viene spostata in una factory dedicata.

Il terzo principio è l'**Information Hiding**: i dettagli implementativi devono essere nascosti dietro interfacce chiare, così che un cambiamento interno non obblighi a modificare i client del modulo.

Il quarto principio è l'**Open/Closed Principle**: le classi devono essere aperte all'estensione ma chiuse alla modifica. Questo principio è supportato soprattutto da Strategy, Factory Method e State.

Infine, l'architettura mira a massimizzare la coesione e minimizzare l'accoppiamento, evitando che una modifica in una classe provochi effetti a catena sull'intero sistema.

6. WIREFRAME

Di seguito viene mostrata una prima configurazione dell'applicazione Music Player tramite schema wireframe

Schermata Principale

The main interface is titled "Music Player" with the subtitle "Create playlists, manage tracks, simulate playback and apply advanced filters". It features a search bar at the top right labeled "Search tracks, artists, playlists..." with a "Clear" button. The interface is divided into several sections: "Playlists" on the left with "New", "Rename", and "Delete selected playlist" buttons; "Track library" in the center with "Add track", "Edit", and "Delete" buttons, containing a table with columns "Title", "Author", "Length", "Genre", and "Year"; "Simulated playback" on the right with a "Now playing" field, a progress bar from "00:00" to "00:00", and "Play", "Pause", and "Skip" buttons; "Advanced filters" with a "Title or author" search, a "Genre" dropdown, "Year from" and "Year to" fields, a "Sort by" dropdown set to "Title (A-Z)", and "Visual tags" for "Favourite", "Explicit", and "New release"; and a "Most played" section at the bottom right. A "Ready" status indicator is at the bottom left, and "Undo" and "Redo" buttons are at the bottom right.

Schermata di Aggiunta/Modifica di un brano

This modal is titled "Music Player" and "Add track". It contains input fields for "Title", "Author", "Length" (with a hint "e.g. 3:45 or 225 (seconds)"), "Genre", and "Year" (with a hint "e.g. 2024"). There is a text area for "Notes / Description (optional)". At the bottom, there are "Visual tags" for "Favourite", "Explicit", and "New release". A "Ready" status indicator is at the bottom left, and "Reset", "Cancel", and "Add track" buttons are at the bottom right.

Schermata Pop-up per l'eliminazione del brano

This pop-up is titled "Delete track" and contains the text "Sei sicuro di voler eliminare il brano?" and "Questa azione non può essere annullata." at the bottom, with "Cancel" and "Delete" buttons.

Sch. pop-up per aggiungere/

This pop-up is titled "New playlist" and contains a "Playlist name" input field with a placeholder "Enter playlist name". At the bottom, there are "Cancel" and "Create playlist" buttons.

7. Diagramma dei Package

Il diagramma seguente rappresenta la **Development View** del progetto *Music Playlist Manager*.

La struttura del codice è organizzata all'interno del package generale **src**, suddiviso in due aree principali: **main**, che contiene il codice dell'applicazione, e **test**, dedicato ai test unitari.

Nel package **main** sono presenti i package principali **view**, **controller**, **service**, **model** e **command**, ciascuno con una responsabilità specifica. Il package **service** rappresenta il livello di business logic e coordina le operazioni tra **controller**, **model** e **command**.

Il package **test** raccoglie invece i test relativi ai componenti principali del sistema.

